

CineMax

Manuale Tecnico

Laboratorio Interdisciplinare A

Autori:

Daniele Cialdini

Daniele di Trapani

Matteo Corti

Pietro Longoni

Versione 1.0

Giugno 2026

1. Installazione	3
1.1 Requisiti di sistema	3
1.2 Setup ambiente	3
1.3 Installazione programma	3
2. Esecuzione ed uso	3
2.1 Setup e lancio del programma.....	3
2.2 Uso delle funzionalità	3
2.3 Data set di test	4
3. Limiti della soluzione sviluppata	5
4. Sitografia/Bibliografia	5
5. Architettura del sistema e scelte progettuali	6
5.1 Suddivisione del codice e Responsabilità delle classi.....	6
5.2 Ereditarietà e gerarchia degli attori.....	6
5.3 Strutture dati e Relazione tra oggetti	7
5.4 L'algoritmo di sicurezza: Hashing Polinomiale	7
5.5 Parsing avanzato: Date, Orari e Regex	7
5.6 Normalizzazione dei dati e Data cleansing	8
5.7 Gestione delle eccezioni e affidabilità.....	8

1. Installazione

1.1 Requisiti di sistema

Per la compilazione e l'esecuzione dell'applicazione Cinemax, il sistema ospite deve soddisfare i seguenti requisiti:

- **Java Development Kit (JDK):** Versione 8 o successiva. Il sistema si avvale sia delle API legacy (java.util.Date) sia del moderno framework introdotto in Java 8 (java.time.LocalDateTime).
- **Sistema Operativo:** Qualsiasi piattaforma supportata da Java (Windows/macOS/Linux).
- **RAM:** Minimo 512 MB.
- **Spazio sul Disco:** Meno di 1 MB per l'eseguibile, più lo spazio necessario all'archiviazione dei file di testo (TXT/CSV).

1.2 Setup ambiente

Verificare la corretta installazione di Java sul sistema aprendo un terminale e digitando:
`java -version`

1. Nel caso in cui il comando non venga riconosciuto, procedere al download di una distribuzione JDK stabile.
2. Predisporre la cartella radice del progetto mantenendo rigorosamente la struttura dei pacchetti logici. Il codice sorgente appartiene al package cinemax
3. Creare all'interno della cartella principale dell'applicazione una sottocartella denominata data/. Questa directory è indispensabile in quanto il software vi cercherà i file di persistenza del database (proiezioni.csv / prenotazione.txt / Utenti.txt)

1.3 Installazione programma

L'applicazione non richiede un installer grafico. L'installazione si effettua compilando il file sorgente tramite la riga di comando. Posizionarsi nella cartella src del progetto ed eseguire il comando: `javac cinemax/*.java`

La compilazione genererà i file bytecode .class all'interno della cartella del pacchetto, completando la procedura di installazione del programma.

2. Esecuzione ed uso

2.1 Setup e lancio del programma

Prima del primo avvio, assicurarsi che nella cartella data siano presenti i file di persistenza (anche vuoti). Per lanciare l'applicazione, posizionarsi nella cartella principale del progetto ed eseguire: `java -cp src cinemax.Cinemax`.

Oppure se la si avvia da un IDE, basterà cliccare sul tasto Run per avviare il tutto.

2.2 Uso delle funzionalità

All'avvio, l'interfaccia a riga di comando mostra il menù principale di benvenuto gestito da un ciclo interattivo:

1. **Login:** Consente l'accesso profilato selezionando esplicitamente il ruolo (Cliente, Bigliettaio, Proiezionista). Il sistema richiede username e password; quest'ultima viene istantaneamente convertita in un codice hash polinomiale positivo (tramite classe Password) per essere confrontata in modo sicuro con le credenziali archiviate
2. **Registrazione nuovo cliente:** Permette ad un utente esterno di inserire anagrafica, username univoco e password, inserendolo nell'archivio utenti
3. **Accesso come Guest:** Permette la navigazione anonima orientata solo alla consultazione dei film e delle proiezioni

A seconda del ruolo autenticato con successo, l'applicazione sblocca i rispettivi sottomenù

- **Cliente Registrato:** Può inserire una nuova prenotazione (il sistema calcola automaticamente il codice univoco PRN-XXXXXXX e decrementa la disponibilità della sala di 200 posti complessivi), visualizzare il proprio storico e cancellare/modificare prenotazioni (consentito solo se l'orario della proiezione non è ancora trascorso)
- **Bigliettaio:** Abilitato alla visualizzazione complessiva delle prenotazioni della giornata odierna e alla ricerca puntuale delle prenotazioni di qualsiasi utente
- **Proiezionista:** Operatore tecnico addetto alla manipolazione del palinsesto. Può inserire un nuovo film/proiezione, aggiorna le date o eliminare una proiezione da file CSV

2.3 Data set di test

Per convalidare i flussi operativi del software, si raccomanda l'uso dei seguenti record di test preimpostati all'interno della cartella data/:

File data/Utenti.txt – Esempio:

Mario,Rossi,mario90,124567,19/05/1990,Milano,Cliente

File data/proiezioni.csv – Esempio:

dataOra,titolo,genere,regista,anno,durata,etaMinima,prezzo

File data/prenotazioni.txt – Esempio:

PNR-A1B2C3D4;mario90;Inception;Christopher Nolan;2026-07-15 21:00:00;2;15.00

3. Limiti della soluzione sviluppata

Il sistema **Cinemax**, pur rispondendo coerentemente alle specifiche di tracciamento e gestione, presenta dei vincoli architetturali intrinseci:

1. **Persistenza concorrente assente:** L'utilizzo di file piatti di testo (.txt e .csv) riscritti integralmente a ogni modifica espone il sistema a potenziali corruzioni dei dati
2. **Sicurezza della crittografia locale:** L'algoritmo introdotto nella classe Password sfrutta una funzione hash polinomiale personalizzata con parametri costanti fissi. Questo meccanismo riduce i conflitti immediati a schermo, ma non è idoneo a contesti di produzione commerciali, dove risulterebbe molto vulnerabile ad attacchi di collisione
3. **Capienza sale statica:** La capienza massima della sala è definita in modo rigido e immutabile tramite una costante statica (capienzaSala = 200) all'interno della classe Proiezioni. Non è configurabile la presenza di sale con capienze differenti per proiezioni contemporanee diverse
4. **Interfaccia Utente Bloccante:** L'interfaccia internamente basata su console testuale sincrona (Scanner) blocca il thread principale ad ogni interazione dell'utente, impedendo l'esecuzione fluida di elaborazione parallele o di notifiche asincrone in tempo reale

4. Sitografia/Bibliografia

- **Oracle Java Documentation:** Riferimenti ufficiali per le classi dei framework Core di Java specificamente per le manipolazioni di struttura dinamiche (java.util.List, java.util.Map, java.util.HashMap) e per il parsing temporale moderno (java.time.LocalDateTime, java.time.format.DateTimeFormatter).
URL: <https://docs.oracle.com/javase/>
- **Java Platform SE Binary IO:** Documentazione relativa alle operazioni di lettura/scrittura sequenziale a buffer su supporti di memoria persistente (java.io.BufferedReader, java.io.BufferedWriter, java.io.FileReader, java.io.FileWriter)
- **Regular Expressions Tutorial (Regex):** Studio del pattern avanzato utilizzato nel metodo caricaDaCSV ("(?=([^"]*" | "*" | "*")*" [^"]*")") per interpretare correttamente le virgole di separazione ignorando quelle incluse tra doppi apici testuali.

5. Architettura del Sistema e Scelte Progettuali

L'applicativo CineMax è stato sviluppato seguendo rigorosamente il paradigma della Programmazione Orientata agli Oggetti (OOP). Il design pattern architetturale scelto si basa su un'alta coesione interna alle classi e un basso accoppiamento, delegando la gestione dell'I/O (Input/Output) a classi dedicate per separare la logica di business dalla persistenza dei dati.

5.1 Suddivisione del Codice e Responsabilità delle Classi (Separation of Concerns)

Un aspetto fondamentale dell'architettura è la netta separazione delle responsabilità (Separation of Concerns) all'interno dei pacchetti di codice. L'applicativo non mescola la logica di visualizzazione a schermo con i dati veri e propri, ma si divide logicamente in due grandi blocchi:

⑩ **Il gestore dell'Interfaccia Utente (CLI):** Tutta l'interazione con l'utente (stampa dei menù a schermo, lettura degli input da tastiera tramite Scanner, cicli infiniti while per la navigazione) è stata centralizzata all'interno di un'unica classe principale (CineMax.java). Questa classe funge da "direttore d'orchestra" per il programma.

⑩ **Il Modello dei Dati:** Tutte le altre classi (Proiezioni, Film, Prenotazione e la gerarchia Utenti) rappresentano il modello puro dei dati. Queste classi non si occupano di gestire l'interfaccia, ma si limitano a memorizzare le informazioni, validare lo stato interno ed elaborare la logica (come la decurtazione dei posti disponibili in sala). Questa divisione rende il codice estremamente modulare: se un domani si volesse sostituire la schermata nera del terminale con un'interfaccia grafica a bottoni (GUI), il codice delle classi del modello dei dati rimarrebbe del tutto invariato.

5.2 Ereditarietà e Gerarchia degli Attori

Per garantire la scalabilità e la corretta gestione dei privilegi, il sistema modella gli attori tramite un solido albero di ereditarietà:

⑩ **Classe astratta Utenti:** Funge da classe base (supertipo) per tutti gli attori. Incapsula i dati anagrafici comuni e le credenziali, proteggendoli con i modificatori di accesso private ed esponendoli tramite metodi getter/setter, nel pieno rispetto del principio di incapsulamento.

⑩ **Polimorfismo e Sottoclassi Concrete:**

⑩ **Guest:** Sottoclasse che rappresenta un utente non loggato. Implementa le sole funzioni di ricerca e lettura delle proiezioni pubbliche, tutelando i dati sensibili.

⑩ **Registrati:** Estende Utenti e implementa la logica di business relativa ai clienti paganti. Gestisce direttamente le operazioni CRUD (Create, Read, Update, Delete) sulle proprie Prenotazioni.

⑩ **Bigliettai e Proiezionisti:** Classi dedicate allo staff tecnico, separate per segregazione delle interfacce. I Proiezionisti hanno i metodi per manipolare lo scheduling e il palinsesto (scrittura su CSV), mentre i Bigliettai hanno permessi di validazione dei codici in cassa. Questa divisione garantisce l'applicabilità del principio **Open/Closed**: in futuro sarà possibile aggiungere una nuova classe (es. Manager) estendendo Utenti, senza dover alterare il codice esistente.

5.3 Strutture Dati e Relazioni tra Oggetti

Per la memorizzazione a runtime dello stato del cinema, il sistema sfrutta il Java Collections Framework, in particolar modo implementazioni dell'interfaccia `java.util.List` (come `ArrayList`), ideali per query di ricerca veloci e aggiornamenti dinamici in memoria.

Le relazioni tra le entità principali sono state modellate seguendo gli standard UML:

⑩ **Associazione Multipla (1-a-N):** La classe Prenotazione funge da entità associativa. Ogni oggetto Prenotazione contiene al suo interno un riferimento diretto a un oggetto Proiezioni e a un oggetto Film, mantenendo il legame referenziale in memoria.

⑩ **Costruttori di Caricamento vs Creazione:** Prenotazione sfrutta l'overloading dei costruttori. Un costruttore genera il codice univoco alfanumerico (usato durante un nuovo acquisto), mentre un secondo costruttore riceve il codice già esistente come parametro, utilizzato esclusivamente in fase di deserializzazione (lettura dal file `prenotazioni.txt`).

5.4 L'Algoritmo di Sicurezza: Hashing Polinomiale

La sicurezza delle credenziali all'interno di `Utenti.txt` è gestita dalla classe `Password`. Aniché salvare i dati sensibili in chiaro (pratica deprecata in ambito ingegneristico), il sistema implementa un algoritmo matematico di **Hashing Polinomiale iterativo**. L'algoritmo elabora la stringa carattere per carattere applicando la formula iterativa: $\text{hash} = (31 * \text{hash} + \text{codice_ascii(carattere)}) \% 1007$. La scelta della **base 31** (un numero primo piccolo) e del **modulo 1007** (un numero primo grande per il dimensionamento della tabella hash fittizia) è una pratica standard mutuata dalla libreria core di Java per garantire una distribuzione uniforme dei valori ed evitare collisioni matematiche. Una volta generato l'hash, il login confronta matematicamente il valore calcolato con quello persistito, rendendo l'autenticazione sicura.

5.5 Parsing Avanzato: Date, Orari e Regex

La persistenza basata su file di testo piatto (CSV) ha richiesto l'implementazione di algoritmi di parsing (deserializzazione) complessi per garantire l'integrità dei dati:

1. **Elaborazione delle stringhe (Regex):** Per leggere il file `proiezioni.csv`, l'applicativo utilizza l'espressione regolare avanzata `"(?=([^\"] *\" [^\"] *)* [^\"] *$)"`. Questo meccanismo permette il metodo `split()` di ignorare in modo selettivo le virgole che si trovano all'interno di titoli racchiusi tra doppi apici (es. "Il buono, il brutto, il cattivo"), evitando che le colonne del CSV si sfalsino causando un'eccezione a runtime (`IndexOutOfBoundsException`).

2. **Manipolazione Temporale API Moderna:** Il controllo temporale delle proiezioni sfrutta la moderna libreria `java.time` (introdotta in Java 8). L'utilizzo congiunto di `LocalDateTime` e `DateTimeFormatter` garantisce che un utente Registrati non possa cancellare o modificare una prenotazione se il controllo cronologico `LocalDateTime.now().isAfter(dataProiezione)` risulta vero, blindando la logica di business.

5.6 Normalizzazione dei Dati e Data Cleansing (trim e replace)

Durante l'estrazione e il confronto dei dati provenienti dal file CSV (come durante le ricerche iterando sull'elencoProiezioni), è stata applicata un'importante logica di **Data**

Cleansing (pulizia del dato) avvalendosi dei metodi `replace("\\", "")` e `trim()`. Spesso, infatti, i file testuali generati contengono apici per raggruppare le parole (aggiunti automaticamente per preservare gli spazi) o possono contenere involontari spazi bianchi (whitespace) creati manualmente o da difetti di parsing. Ad esempio, un titolo memorizzato come "Inception" fallirebbe una ricerca dell'utente che digita Inception per colpa degli spazi e delle virgolette non visibili. Ripulendo a monte le stringhe (`String titoloPr = pr.getTitolo().replace("\\", "").trim();`), si realizza un processo di *normalizzazione* che garantisce un matching sicuro, immunitario da difetti di formattazione del file di origine e migliorando sensibilmente l'User Experience e la stabilità del software.

5.7 Gestione delle Eccezioni e Affidabilità (Robustness)

L'intera architettura è protetta (fault-tolerant) tramite un blocco sistematico delle eccezioni. Le operazioni di I/O (lettura file tramite `BufferedReader` e scrittura tramite `BufferedWriter`) gestiscono accuratamente le `IOException`. Le conversioni di formato (es. `Integer.parseInt`) intercettano silenziosamente le eccezioni, impedendo crash fatali qualora il file persistente risultasse corrotto, assicurando che la *Command Line Interface* (CLI) ritorni sempre al ciclo del menù principale per avvisare l'utente.